

Gacrux software

Ver. 1.9, aligned to TieBreakServer 1.9.57 (2026-07-21)

Original author: Otto Milvang

Ver. 1.9 update (August 3, 2026): Paul Hampton - aligned this specification to the implementation (code is authoritative)

Introduction

The Gacrux software is a collection of tools and shared data structures designed to support Tournament Handler Programs (THPs) for chess events. Its core purpose is to provide a consistent, transparent, and verifiable representation of tournament data-covering players, rounds, pairings, results, rankings, and tiebreaks-throughout the lifecycle of an event.

At the heart of Gacrux is a unified internal structure that can be serialized to and from JSON. In this document that structure is called the **Json Chess** representation. On the command line and in file I/O the format name is **JSON** (file extensions `.jch` and `.json` both map to JSON). The structure is designed to be human-readable, machine-processable, and easy to extend.

Gacrux includes command-line programs for pairing generation and verification, tiebreak calculation, ranking checks, and synthetic tournament generation. These tools operate on JSON files or import and export common tournament formats, including FIDE TRF and TournamentService (TS) files.

Every operation produces structured output that includes status codes, diagnostics, and-when requested-detailed explanations of pairing and tiebreak decisions.

Json Chess (JSON)

The Json Chess file defines the canonical data structure used by the Gacrux software. A file encapsulates the complete state of a tournament, including identifiers, player and team data, rounds, pairings, results, rankings, and tiebreak information.

CLI and API format identifiers are **JSON**, **TRF**, and **TS**. The string JCH is not accepted as an `-f / -F` value; use JSON.

Each program records origin, options, status, and results in a consistent JSON envelope so transformations can be inspected, validated, and reproduced. The definition is versioned; backward-compatible extensions may be introduced as needed.

Programs

Common program syntax

Command-line options are divided into common options (I/O, encoding, verbosity, tournament selection) and program-specific options.

```
python programname.py [options]
```

Option	Description	Default
<code>-i, --input-file <filename></code>	Path to input file. "-" = stdin.	stdin (-)
<code>-o, --output-file <filename></code>	Path to output file. "-" = stdout.	stdout (-)
<code>-f, --input-format <format></code>	JSON - TRF - TS. Other values are rejected (status 403).	TRF
<code>-F, --output-format <format></code>	JSON - TRF - TXT. If <code>-d</code> is set, text output is used regardless of <code>-F</code> . TS output is not implemented.	JSON
<code>-b, --encoding <encoding></code>	Python codec name (ascii, utf-8, cp1252, ...).	JSON->utf-8; TRF->latin1; TS->ascii
<code>-e, --tournament-number <n></code>	Tournament index in multi-event files (1-based). 0 = all / passthrough where supported.	1

Option	Description	Default
-n, --current-round <n>	Round override; meaning depends on the program (-1 = use file / program default).	-1
-N, --number-of-rounds <n>	Overrides tournament numRounds when > 0.	0
-G, --game-score <key:val ...>	Override game point system (KEY:val or KEY=val, comma-separated groups).	see score table
-M, --match-score <key:val ...>	Override match point system.	see score table
-c, --check	Check mode (count flag). Meaning is program-specific.	off
-r, --rank	Sort text output in rank order (tiebreakchecker).	off
-d, --delimiter <text>	Force text output. T=tab, B=blank, S=semicolon, C=comma, @=status line, or literal text.	unset -> JSON
-D, --decimal-point <text>	Decimal mark in text output. C=comma, P=point, or literal.	unset
-x, --experimental <list>	Experimental keywords (e.g. "weighted", "time", "DUMP").	[]
-v, --verbose	Progress / debug (count: -v, -vv, ...).	0
-V, --version	Print version string and exit.	off

Note: argparse default for -f is always TRF. Extension-based detection via `helpers.getFileFormat()` is used by chessserver when resolving `input_filename`, not by the standalone CLI parsers.

Score system defaults (-G / -M)

Key	Meaning	Default game (-G)	Default match (-M)
W	Win	1.0	2.0
D	Draw	0.5	1.0
L	Loss	0.0	0.0
Z	Zero point bye	0.0	0.0
P	Pairing allocated bye	W	D
A	Adjourned	D	D
F	Full point bye	W	W
H	Half point bye	D	D
U	Unknown	Z	Z
FG / HG / ZG / PG	Match game-points for F/H/Z/P	-	W* / D* / Z* / P*

Example (individual PAB as draw): -G P:D. X* means game points for X times team size.

JSON output envelope

Common JSON output has a header, status, and a result object keyed by program (pairingResult, tiebreakResult, ...). Mandatory fields are shown in bold in spirit; all of the following are always present on success paths.

Key	Type	Description
filetype	String	Pairing - Tiebreak - Event - ...
version	String	Envelope schema version (currently 1.0)
published	String	Creation date-time (YYYY-MM-DD HH:MM:SS)
origin	String	Program name and version
options	Object	Parsed CLI / request options
status	Object	code, info[], error[]
<filetype>Result	Object	Operation result (not always an array)

Status object

code	Meaning
0	OK; with -c, check passed

code	Meaning
1	OK path with check failed
2	OK but no legal pairing (empty pairs)
200-202	Same semantics as 0-2 for web contexts where used
400-499	Client / input errors (e.g. 401 read, 403 format, 405 empty)
500-599	Internal / program errors

```
{
  "filetype": "Pairing",
  "version": "1.0",
  "published": "2026-04-06 16:30:57",
  "origin": "pairingchecker ver. 1.9.57",
  "options": { "input_file": "test1315.trf", "input_format": "TRF", ... },
  "status": { "code": 0, "info": [], "error": [] },
  "pairingResult": { "rules": "2026-03-01", "round": 3, "pairs": [[1,6], ...] }
}
```

pairingchecker.py

python pairingchecker.py [options]

Generate, analyse, and verify pairings according to the applicable pairing rules. In analysis mode the program explains the pairing already present for a round. In pairing mode it computes the best pairing. In check mode it compares the computed pairing to the tournament file.

Options (in addition to common options)

Option	Description	Default
-m, --method <name>	Pairing system. Legal engines: dutch, berger. Suffixes after '-' set primary score: dutch-mp / dutch-gp (also match/game). Token double raises maxMeets.	[] -> tournament pairingSystem, else dutch
-n, --current-round <n>	Round to pair/analyse/check. Defaults: -p -> next round; -a -> last paired; -c -> all played rounds.	-1
-p, --pairing	Compute pairing (count flag).	off
-a, --analysis	Analyse existing pairing (count flag).	off
-c, --check	Compare pairing to analysis / file. With -p/-a adds detail; alone summaries rounds.	off
-t, --top-color <w b>	Colour for top competitor in round 1.	blank -> derived / random path
-u, --unpaired <cid ...>	Competitors not to be paired (marked absent) for the run.	[]
-K, --maxmeets <n>	Maximum meetings between the same pair. Effective value is at least 1.	0 -> clamped to >=1
-T, --exchange <a-b ...>	Test mode: rewrite pairs then re-analyse (use with -c -a -p -n).	[]
-x, --experimental <...>	weighted (reference algorithm), time, fakerank, TPN, QMM, DUMP, ...	[]

Json output - pairingResult

The top-level result key is pairingResult. It is a **single object** (not an array).

Pairing / analysis mode (without -c)

Key	Type	Description
rules	String	Rule date string used by the engine
round	Int	Round paired or analysed
pairs	Array of [w,b]	Computed pairs; b=0 is pairing-allocated bye

```
"pairingResult": {
  "rules": "2026-03-01",
  "round": 3,
  "pairs": [[1,6],[11,2],[5,10],[17,15],[7,3],[4,9],[12,16],[8,13],[14,0]]
}
```

Status code 0 if pairs is non-empty, else 2.

Check mode (-c)

Key	Type	Description
rules	String	Rule date
check	Boolean	True if all round checks passed
roundpairing	Array	Per-round detail objects

roundpairing element

Key	Type	Description
round	Int	Round number
pairs	Array	Checker pairing

Key	Type	Description
current	Array	Pairing from tournament file
check	Boolean	pairs == current
pairing	Array	Per-scorebracket pairing information
analysis	Array	Per-scorebracket analysis of file pairing
competitors	Array	Competitor snapshots used by the engine
level2score	Array	Scorelevel -> points; PAB is -1.0 at level 0

Pairing information (score bracket)

Key	Type	Description
scorelevel	Int	Internal bracket index
competitors	Array of int	Competitor ids in bracket
pairs	Array of objects	Pair details
downfloaters	Array of int	Downfloated competitors
remaining	Array of int	Remaining competitors
quality	Object	Named quality weights (QC6...HO5, ...); lower is better
bsne / bsno	Array	BSN lists when present
pab	Boolean	True if PAB level

Pair details (dutch)

Key	Type	Description
ca / cb	Int	Colour-independent competitor ids (cb >= ca)
sa / sb	Int	Scorelevels
w / b	Int	White / black ids; b=0 for PAB
canmeet	Boolean	Legal meeting
played	Int	Prior meetings
quality	Object	Per-pair quality weights
colorrule	String	Colour assignment rule id
board	Int/String	Board number
mode	String	Pairing mode marker

Competitors (pairing snapshot)

Key	Type	Description
cid	Int	Competitor id
pts / acc	Float	Points / points + acceleration
rfp	Boolean	Ready for pairing (present)
hst / met	Object	History / opponents met
num / rip	Int	Played games / rounds paired
cod / cop / csq	...	Colour difference / preference / sequence
flt	Int	Float bits (8=df, 4=uf previous; 2/1 two rounds earlier)
top	Boolean	Top scorer
scorelevel	Int	Score level

Text output

python pairingchecker.py -i <input> -o <output> -p -dT prints the number of pairs, then lines w b (compatible with JaVaFo / bbpPairings style). -c -dT prints per-round headers and differences; -c -d@ prints

status code 0 or 1.

tiebreakchecker.py

python tiebreakchecker.py [options]

Calculate and verify tiebreak values and ranking order. Calculate mode computes tiebreaks and ranks; check mode also compares ranks to the tournament file.

Options (in addition to common options)

Option	Description	Default
-n, --current-round <n>	Rounds included in the calculation.	-1 -> all rounds in file
-t, --tiebreak <list>	Rank-order specifiers. If empty and the tournament has rankOrder, that list is used.	[]
-p, --pre-determined	Treat as pre-determined (round-robin) pairing for unplayed-game rules.	off -> params.pre_determined
-s, --swiss	Force Swiss unplayed-game rules.	off
-u, --unrated <rating>	Rating for unrated players in rating TBs.	unset

Tiebreak list syntax

TB :pp /Mn /opt - TB name; optional :MP/:GP (and ESB :MM| :MG| :GM| :GG); modifiers /C1 /C2 /M1 /M2 /L±n /Kx /P /F /U1400 /V1 /V2024 /V2 /V2026 /R /S.

Point systems and main tiebreaks

Name	Modifiers	Notes
PTS / MPTS / GPTS / MPVGP	:MP :GP	Points / match / game / primary flip
DE	/P	Direct encounter
WIN / WON / BPG / BWG	:MP	Wins / games won / black played / black won
PS	:MP :GP /C...	Progressive score
REP (GE deprecated alias)		Rounds elected to play
STD		Standard score helper
BH / FB / SB / AOB	:MP :GP /C /M /P /F	Buchholz family; FB == BH/F
KS	:MP :GP /L...	Koya
ARO / TPR / PTP / APRO / APPO	/C /M /U...	Rating performance family
BC / TBR / BBE		Board-count family (teams)
ESB / EMMSB / EMGSB / EGMSB / EGGSB	:MM... /C /P	Extended Sonneborn-Berger
EDE	/P	Extended direct encounter
SSSC	/Kx /P /F	Score strength combination
SNO / RANK / RTG / RND / VUR / ABH / AFB	/P /R	Helpers / order keys
NUM / COP / COD / CSQ / ACC / FLT / RFP / TOP		Pairing diagnostics helpers

Version modifiers (/V...)

Modifier	Internal ver	Rules date
/V0 or /V2022	0	2022-01-01
/V1 or /V2024	1	2024-08-01
/V2, /V2026, or other	2	2026-03-01
/V (bare)	2	latest (max key)

tiebreakResult

The result key is `tiebreakResult`, a single object:

Key	Type	Description
rules	String	Rule date (from ver)
round	Int	Rounds used
tiebreaks	Array	Parsed TB descriptors (order, name, pointtype, modifiers, precision)
competitors	Array	Per-competitor results
check	Boolean	Present when -c

TB modifiers object (typical keys)

Key	Type	Description
ver	Int	0/1/2 rule set
rev	Boolean	Sort direction flip relative to TB default
cutlow / cuthigh	Int	Cut / median cuts
plim / nlim	Number	KS limits (percent / score)
unrated	Int	Substitute rating
predetermined	Boolean	Unplayed treated as played
swiss	Boolean	Swiss unplayed rules
foremode	Boolean	Fore Buchholz
urd / exchange / vun	Boolean	Experimental / special flags

competitor object

Key	Type	Description
cid	Int	Competitor id
rank	Int	Computed rank
tiebreakScore	Array	Values aligned with tiebreaks[]
tiebreakDetails	Array	Present with -c; per-TB contribution detail
boardPoints	Object	Team + -c only: board number -> points

There is no separate boardCount field in JSON; BC is a tiebreak whose score is derived from boardPoints.

Text output

-d T (or S/C/B) prints a header row StartNo/Rank (or Rank/StartNo with -r) plus each -t token, then one row per competitor. With -c an extra Check: True/False line is written. -d@ prints the status code only.

tournamentgenerator.py

```
python tournamentgenerator.py [options]
```

Supporting tool for testing and development. Generates synthetic tournaments with controlled statistics using the same pairing engines as pairingchecker, exported as TRF.

Option	Description	Default
-g, --generate ...	1-3 ints: [count] or [start count] or [start count step]. Two-arg form: range(start, start+count).	0 1000
-o, --output-file	%d in the path -> zero-padded tournament number.	required for files
-p, --players	Number of competitors.	40
-T, --members	Players per team (playing).	1
-m, --method	dutch or berger (and -suffixes as in pairingchecker).	dutch
-t, --top-color	Accepted; generator currently alternates by file number.	blank
-R, --rating [top step sigma]	Rating model; omitted -> heuristics by field size.	[]
-S, --statistics [zpb hpb forfeited]	Rates of ZPB, HPB, forfeits. RR forces zpb=hpb=0.	0.01 0.05 0.02
-a, --acceleration	Baku acceleration.	off
-K, --maxmeets	Max meetings (clamped like pairing).	0
-n / -N	Round count via common options.	see common

```
python tournamentgenerator.py -g 500 -n 9 -p 300 -o c:\files\myfilename%d.trf
```

chessserver.py

```
python chessserver.py
```

Stdin/stdout JSON service for web deployment. Services: convert, pairing, tiebreak. Unknown service falls back to convert. No authentication; put a reverse proxy in front. Host paths from the client are ignored; only in-memory payloads are accepted (max request/payload 10 MiB).

Request

The body is a JSON object with a **command** property holding the options:

```
{
  "command": {
    "service": "convert" | "pairing" | "tiebreak",
    "input_format": "TRF" | "TS" | "JSON",
    "input_filetype": "TRF" | "TS" | "JSON",
    "input_filename": "optional name for extension sniffing",
    "base64": "<base64 tournament file>" | [...],
    "jch": "<raw Json Chess text>",
    "data": "<base64>" | ["<base64 lines>"],
    "encoding": "ascii",
    "tournament_number": 1,
    "current_round": -1,
    "number_of_rounds": 0,
    "check": false,
    "tiebreak": ["PTS", "DE", "BH/C1"],
    "pre-determined": false,
    "swiss": false,
    "unrated": null,
    "pairing": true,
    "analysis": false,
    "method": "dutch",
    "top_color": "w",
    "maxmeets": 1,
    "unpaired": [],
    "rank": false,
    "game_score": {},
    "match_score": {},
    "experimental": [],
  }
}
```

```

    "verbose": 0
  }
}

```

At least one of base64, jch, or data is required. If input_format is omitted, input_filetype is used, else getFileFormat(input_filename), else TRF. Server default encoding is ascii (so TRF over the server defaults to ascii, not latin1). Allowlisted option names use underscores except pre-determined (hyphen), matching the CLI long option spelling.

Response

JSON envelope identical in shape to the corresponding CLI program output (convert -> event JSON; pairing -> pairingResult; tiebreak -> tiebreakResult). When base64 or jch was used, stdout may be prefixed with a Content-Type: application/json header.

helpers.getFileFormat

Extension	Format id
.jch, .json	JSON
.trf, .trfx, .txt	TRF
.trx	TS
other / missing	JSON

This document describes the behaviour of TieBreakServer 1.9.57. Where earlier drafts differed (for example format name JCH as a CLI token, pairingResult as an array, or chessserver top-level options without a command wrapper), the implementation is authoritative. The Ver. 1.9 alignment of this specification to the code was made by Paul Hampton.